

Instytut Informatyki, Politechnika Poznańska

Klaster obliczeniowy - instrukcja użytkownika

Spis treści

[Aktywacja dostępu do klastra](#)

[Udostępniane zasoby](#)

[Uruchamianie obliczeń](#)

[Węzeł dostępowy](#)

[Katalog domowy](#)

[System operacyjny](#)

[Instalacja oprogramowania](#)

[Sesja interaktywna](#)

[Przykładowa instalacja - Conda + PyTorch](#)

[Zlecanie zadań](#)

[Krótkie zadania w trybie interaktywnym](#)

[Zlecanie zadań w trybie wsadowym](#)

[Anulowanie zadania](#)

[Monitorowanie stanu systemu i zadań](#)

[Lista dostępnych węzłów i ich stan](#)

[Sprawdzenie stanu kolejki](#)

[Sprawdzanie historii zadań](#)

[Wyświetlanie limitów zasobowych](#)

[Polecenie sacctmgr](#)

[Polecenie scontrol \(dla zaawansowanych\)](#)

Aktywacja dostępu do klastra

W celu uzyskania dostępu do klastra należy zgłosić zapotrzebowanie e-mailowo na adres:

obliczenia@cs.put.poznan.pl

W zgłoszeniu należy zawrzeć następujące informacje:

Dane osoby prowadzącej obliczenia	Imię i nazwisko
	Adres e-mail
	Nr albumu / indeksu (dot. studentów)
	Jednostka organizacyjna /Wydział /Kierunek
Dane opiekuna naukowego (dotyczy studentów i doktorantów)	Imię i nazwisko
	Adres e-mail
Cel obliczeń (wskazać wszystkie właściwe)	Komercyjny

	Niekomercyjny
Dane projektu / projektów (w przypadku prowadzenia obliczeń dla więcej niż jednego projektu prosimy o wyszczególnienie wszystkich) UWAGA: w przypadku rozpoczęcia nowego projektu, na poczet którego będą prowadzone obliczenia należy zgłosić ten fakt na ww. adres.	Nazwa projektu / Tytuł pracy dyplomowej
	Akronim projektu (opcjonalnie)
	Numer projektu instytucji finansującej (np. NCBiR, jeśli dotyczy)
	Numer umowy w EOD (jeśli dotyczy)
	Źródło finansowania (jeśli dotyczy) np. 0311/SBAD/07xx
	Typ wykorzystania zasobów w ramach projektu (komercyjny / niekomercyjny)
	Czy wymagany dostęp do GPU? (Tak/Nie)
Wymagany okres dostępu (jeśli różny dla każdego projektu, to prosimy o wskazanie per projekt)	daty: od - do

W odpowiedzi na zgłoszenie przesłane zostaną informacje dostępowe (ang. login) i nazwy kont rozliczeniowych (ang. account) na potrzeby systemu kolejkowego.

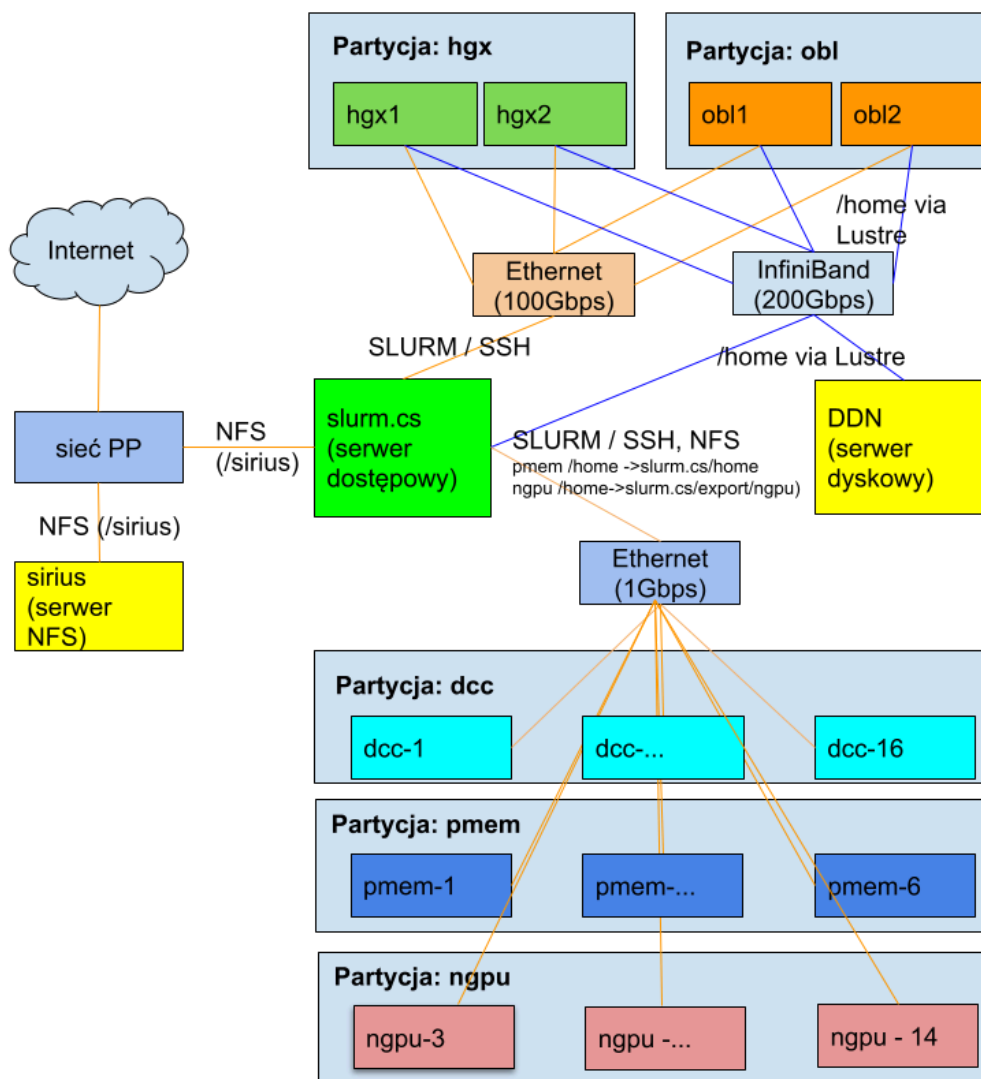
Dla każdego projektu utworzone zostaną indywidualne konta rozliczeniowe, do których przypisani zostaną wskazani użytkownicy. Każdy użytkownik może należeć do więcej niż jednego konta rozliczeniowego, oraz do każdego konta może być przypisany więcej niż jeden użytkownik.

W przypadku prowadzenia obliczeń komercyjnych szczególnie istotne jest wskazanie właściwego konta rozliczeniowego (ang. account) przy zlecaniu zadań.

Wszelkie dane osobowe i informacje są zbierane tylko i wyłącznie w celu umożliwienia dostępu do serwerów i należytego rozliczenia się z instytucjami finansującymi. Dane osobowe są przetwarzane zgodnie z zasadami obowiązującymi na uczelni: <https://www.put.poznan.pl/ochrona-danych-osobowych>

Udostępniane zasoby

Aktualnie w klastrze udostępniane są trzy partycje obliczeniowe: hgx, dcc i pmem na poniższym schemacie zaprezentowano topologię połączeń udostępnionych serwerów i zasobów dyskowych.



Udostępnione serwery charakteryzują się poniższymi parametrami:

Nazwa Partycja	Model serwera	Model procesora	Liczba procesorów	Typ GPU	RAM	Zasoby dyskowe
hgx[1-2] hgx	FormatServer THOR EHG21 (Supermicro + NVidia HGX)	AMD EPYC 7543 32-Core Processor	2CPU, 64 rdzenie, 128 wątków	8xA100-80 per server	2TB	/raid - 28TB SSD(lokalne), /home - 236TB na współdz. macierzy DDN (4x InfiniBand 200GbE)
obl[1-2] hgx	FormatServer THOR E221 (Supermicro)	AMD EPYC 7702 64-Core Processor	2CPU, 128 rdzeni, 256 wątków	brak	512GB	/raid - 8.7TB SSD (lokalne), /home - 236TB na

Nazwa Partycja	Model serwera	Model procesora	Liczba procesorów	Typ GPU	RAM	Zasoby dyskowe
						współdz. macierzy DDN
dcc-[1-16] dcc	Dell PowerEdge 860/R200	8×Intel Xeon X3230@2.66GHz, 8×Intel Xeon L3360@2.83GHz	1CPU, 4 rdzenie	brak	4/8 GB	/home - 236TB na współdz. macierzy DDN udostępnionej po NFS (interfejs 1GbE) za pośrednictwem serwera dostępowego
pmem-[1-6] pmem	Inspur NF5280M5	Intel Xeon Gold 6252N@2.30GHz	2CPU, 24 rdzeni, 96 wątków	brak	192GB PMem	/home - 236TB na współdz. macierzy DDN
ngpu-[3-14]	PC	Intel(R) Core(TM) i9-13900K	1 CPU, 24 rdzenie (8 wysokowydajnych, 16 oszczędnych), 32 wątki	NVIDIA GeForce RTX 4080, 16GB* *węzeł 5 jest wyposażony w kartę RTX 4090 24GB	64GB	/raid - dysk 1TB NVMe UWAGA: zaleca się korzystanie z tego dysku w toku obliczeń /home - 10TB na współdz. zasobie udostępnionym po NFS (interfejs 1GbE) z serwera dostępowego - katalog na serwer dostępowym to /export/ngpu

Katalogi domowe /home są odpowiednio:

- dla partycji obliczeniowej `hgx` i `obl` na współdzielonym zasobie (system plików Lustre, 236TB przestrzeni dyskowej) podłączonym za pomocą szybkiej sieci InfiniBand. Na każdym z serwerów w partycji `hgx` jest dodatkowy katalog `/raid` (RAID-0 na szybkich dyskach NVMe), który można wykorzystać do przechowywania np. wyników pośrednich.
- dla partycji `dcc` i `pmem` na tym samym zasobie co powyżej, ale zasób ten jest udostępniany poprzez serwer pośredniczący via NFS,
- dla partycji `ngpu` katalogi domowe znajdują się na serwerze dostępowym (slurm) w katalogu `/export/ngpu` i są udostępniane za pomocą NFS-a po sieć gigabitowej, zaleca się korzystanie z dysków lokalnych (NVMe) w toku prowadzenia obliczeń - dostępny jest katalog `/raid/`.

Ponadto na serwerze dostępowym udostępniane są zasoby z serwera `sirius.cs.put.poznan.pl`, czyli serwera, na którym znajdują się katalogi domowe użytkowników komputerów w laboratoriach Instytutu Informatyki oraz serwera `polluks` (aka `unixlab`). Zasoby te są dostępne jako podkatalogi katalogu `/sirius`.

Maszyny w klastrze `hgx` są połączone wewnętrzną siecią 100GbE i siecią Infiniband. Sieć Infiniband obsługuje dostęp do zasobów dyskowych.

Maszyny w klastrach `dcc` i `pmem` są połączone siecią 1GbE.

Serwery `pmem-[1-6]` wyposażone są w moduły pamięci trwałej PMem (*persistent memory*).

Uruchamianie obliczeń

Zadania obliczeniowe zleca się z [węzła dostępowego](#) za pomocą systemu kolejkowego SLURM. Informacje szczegółowe zawarto w kolejnych sekcjach niniejszego dokumentu.

Węzeł dostępowy

Logowanie na węzeł dostępowy odbywa się za pomocą klienta SSH i konta LDAP nadanego przez Instytut Informatyki, po uprzedniej aktywacji dostępu (vide pkt. [Aktywacja dostępu do klastra](#)).

Nazwa i adres serwera dostępowego:	<code>slurm.cs.put.poznan.pl</code> (150.254.30.245) <code>aka svradmin.cs.put.poznan.pl</code>
---	--

Przykładowa komenda logowania za pomocą klienta ssh:

```
$ ssh <login>@slurm.cs.put.poznan.pl
# <login> oznacza nazwę konta danego użytkownika
```

W systemie Windows można skorzystać z klienta PuTTY lub włączyć opcjonalnego klienta dostępnego z systemem.

Na serwerze dostępowym zainstalowany jest system kolejkowy SLURM, umożliwiający [zlecanie zadań](#) obliczeniowych. Przed zleceniem zadań konieczne może być doinstalowanie wymaganego oprogramowania, co można uczynić za pomocą sesji interaktywnej – vide sekcja [Instalacja oprogramowania](#).

Katalog domowy

Katalog domowy (`/home/<login>`) dostępny na węźle dostępowym, jest współdzielony między węzłami klastra.

Dane z serwera `polluks/sirius/unixlab` są dostępne w katalogu `/sirius/<login>`.

Zaleca się przekopiowanie danych z katalogów domowych serwera `sirius` na szybsze zasoby dyskowe dostępne na węźle dostępowym (tj. do własnego katalogu domowego na węźle dostępowym).

W przypadku partycji `ngpu` zaleca się skopiowanie danych na lokalny dysk `/raid`.

System operacyjny

Na klastrze zainstalowane są systemy z rodziny Linux: w przypadku klastra `hgx` jest to Ubuntu 22.04 LTS, w przypadku klastra `DCC` jest to openSUSE Leap 15.4.

Instalacja oprogramowania

W celu instalacji oprogramowania do wykorzystania na węzłach obliczeniowych za wyjątkiem `ngpu` należy przeprowadzić instalację w przestrzeni użytkownika (do katalogu domowego użytkownika). Najlepiej zainicjować sesję interaktywną z wybranym węzłem obliczeniowym na wybranej partycji obliczeniowej. W przypadku `hgx` może to być np. `hgx1` (jeśli będziemy prowadzić obliczenia na GPU) lub `obl1` (w przypadku braku potrzeby wykorzystywania GPU). W przypadku partycji `dcc` np. `dcc-1`. Dla partycji `ngpu` instalacja oprogramowania wygląda analogicznie, z dokładności do faktu, że katalog domowy na serwerze dostępowym znajduje się w katalogu `/export/ngpu`. Katalog domowy jest dostępny na wszystkich węzłach w ramach danej partycji, a tym samym oprogramowanie się w nim znajduje również. Uruchomienie sesji interaktywnej nastąpi w momencie, kiedy dostępne będą zasoby obliczeniowe umożliwiające wykonanie zadania, jeśli w danym momencie brakuje takich zasobów, to zadanie jest kolejkowane, o czym użytkownik jest informowany stosownym komunikatem. Biblioteki języka Python najlepiej instalować z wykorzystaniem środowiska `conda`.

W uzasadnionych przypadkach istnieje możliwość skorzystania ze środowiska `Docker`, ale ta funkcjonalność jest w fazie testów/integracji z systemem kolejkowym – w przypadku konieczności skorzystania z tej opcji konieczny jest kontakt z administratorami.

Sesja interaktywna

Sesję interaktywną z węzłem obliczeniowym uruchamiamy w następujący sposób:

- Sesja na węźle `hgx1`, z dostępem do 1 karty GPU:
`$ srun -p hgx -w hgx1 --gres=gpu:1 --pty /bin/bash -l`
- Sesja na węźle `obl2`:
`$ srun -w obl2 --pty /bin/bash -l`
- Sesja na węźle `dcc-1`:
`$ srun -p dcc -w dcc-1 --pty /bin/bash -l`
- Sesja na węźle `ngpu-3`:
`$ srun -p ngpu --gres=gpu:1 -w ngpu-3 --pty /bin/bash -l`

Przykładowa instalacja - Conda + PyTorch

- Po nawiązaniu [sesji interaktywnej](#) pobieramy i instalujemy `conda` (<https://repo.anaconda.com/>) w odpowiadającej nam konfiguracji zgodnie z instrukcją: <https://www.anaconda.com/docs/getting-started/miniconda/install#linux-2>

```
$ mkdir -p ~/miniconda3
$ wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -
O ~/miniconda3/miniconda.sh
$ bash ~/miniconda3/miniconda.sh -b -u -p ~/miniconda3
$ source ~/miniconda3/bin/activate

2) Tworzymy nowe wirtualne środowisko w conda
$ conda create -n PYTORCH python
$ conda activate PYTORCH

3) Instalujemy PyTorch (vide: https://pytorch.org/get-started/locally/)
$ pip install torch torchvision torchaudio

4) Sprawdzamy czy działa (pozostając w środowisku conda PYTORCH)
(pytorch) login@ngpu-3:~$ python3
Python 3.14.0 | packaged by Anaconda, Inc. | (main, Oct 22 2025,
09:06:03) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import torch
>>> x = torch.rand(5, 3)
>>> print(x)
```

```

tensor([[0.5745, 0.2404, 0.8054],
        [0.3255, 0.3087, 0.1526],
        [0.5601, 0.4147, 0.3221],
        [0.1270, 0.3042, 0.5865],
        [0.8774, 0.6232, 0.9265]])
>>> torch.cuda.is_available()
True
>>>

```

- 5) Oprogramowanie zostało zainstalowane, możemy zakończyć sesję pythona (quit()) i sesję interaktywną (exit).

Zlecanie zadań

Zadania zlecane są do systemu kolejkowego, który je szereguje z użyciem kolejki priorytetowej. W udostępnianym klastrze wykorzystuje się domyślny algorytm szeregowania Slurm-a (sched/backfill), z włączonym mechanizmem wypełnianie wolnych zasobów krótszymi taskami o mniejszych wymaganiach zasobowych. W przypadku zadań (tasków) interaktywnych niekiedy konieczne może być oczekiwanie na zwolnienie zasobów - zadania interaktywne są kolejgowane w ten sam sposób, co zadania interaktywne. Czas oczekiwania jest zależny od liczby zadań w kolejce i bieżącego obciążenia klastra czy też konkretnego węzła. Węzły dcc mają włączone oszczędzanie energii, więc w ich przypadku oczekiwanie może zostać wydłużone o czas potrzebny na uruchomienie zadania.

Limity zasobowe wyspecyfikowane w wierszu poleceń są weryfikowane w trakcie zlecania zadania i egzekwowane w momencie uruchomienia po stronie węzłów obliczeniowych z wykorzystaniem mechanizmu cgroups.

W klastrze dcc alokowane są całe węzły, w klastrze hgx alokowane są rdzenie procesora i karty graficzne.

Domyślna liczba kart graficznych przypisanych do zadania (task) wynosi zero, co oznacza, że w przypadku chęci skorzystania z GPU trzeba wskazać wprost liczbę niezbędnych kart.

Domyślny, aktualnie ustawiony, **maksymalny czas obliczeń to 24h na zadanie (job)**, wartość domyślną można zmienić za pomocą opcji -t (vide poniżej). Po upływie zdefiniowanego maksymalnego czasu obliczeń, slurm inicjuje przerwanie wykonywania zadania poprzez wysłanie sygnału SIGTERM, który można przechwycić i obsłużyć we własnym programie, a następnie, po 60 minutach, sygnału SIGKILL.

Krótkie zadania w trybie interaktywnym

Do zlecania krótkich zadań, zwłaszcza w trybie interaktywnym, można wykorzystywać polecenie srun np.:

```

$ srun -A <nazwa-konta-rozliczeniowego> -p hgx -nl -cl hostname
$ srun -w hgx1 -A ai -p hgx -nl hostname
$ srun -w hgx1 -A ai -p hgx -nl --gres=gpu:1 nvidia-smi

```

Podstawowe parametry:

- w - nazwa węzła w klastrze
- A - nazwa konta rozliczeniowego
- p - nazwa partycji
- nx - uruchom x identycznych tasków jednocześnie
- cx - użyj maks. x procesorów per task
- t - maksymalny czas obliczeń podany w formacie DD-GG:MM:SS gdzie DD, GG, MM, SS oznaczają odpowiednio żadaną liczbę dni, godzin, minut i sekund na zadanie
- Nx - uruchomienie zadania na co najmniej x węzłach
- mem - ilość pamięci operacyjnej dla całego zadania na węzeł obliczeniowy np. 1GB
- gres=gpu:x - x oznacza liczbę rezerwowanych kart GPU, maksymalna liczba kart to 8 na węzeł hgx (łącznie 16 w klastrze)

Uwaga: nie podanie parametru `--gres=gpu:x` implikuje niedostępność kart GPU dla tak zleconego zadania

Zlecanie zadań w trybie wsadowym

Do zlecania dłuższych zadań należy używać `sbatch` (zamiast `srun`), przykładowe wywołanie wygląda tak:

```
$ sbatch -w hgx1 -nl -A ai --gres=gpu:1 mybatch.sh
```

gdzie `mybatch.sh`, to skrypt uruchamiający zadanie, który jest zwykłym skryptem powłoki np.:

```
#!/bin/bash
hostname
```

zamiast wpisywać część parametrów wywołania `sbatch` w wierszu polecenia możemy je wprowadzić do skryptu np. w poniższy sposób:

```
#!/bin/bash
#SBATCH -w hgx1
#SBATCH -nl
#SBATCH -A ai
#SBATCH --gres=gpu:1
#SBATCH --time=00:05:00
hostname
```

wówczas zlecenie zadania się upraszcza:

```
$ sbatch mybatch.sh
```

Innym przykładem skryptu (`mybatch.sh`) jest skrypt uruchamiający program w Pythonie z PyTorch:

```
#!/bin/sh
conda run -n PYTORCH python ./examplePyTorch.py
w tym przykładzie konieczne jeszcze jest utworzenie skryptu ./examplePyTorch.py w katalogu domowym np.:
#examplePyTorch.py
import torch
import numpy as np
if torch.cuda.is_available():
    device = torch.device("cuda")
    print("Cuda on")
tensor = torch.ones(4, 4, device=device)
print(f"First row: {tensor[0]}")
print(f"First column: {tensor[:, 0]}")
print(f>Last column: {tensor[:, -1]}")
tensor[:,1] = 0
print(tensor)
```

Wynik powyższego polecenia (`sbatch`) zapisze się do pliku `slurm-JOBID.out`

Anulowanie zadania

Zadanie można anulować poleceniem `scancel` - identyfikator zadania najprościej zidentyfikować poprzez sprawdzenie stanu kolejki ([squeue](#)):

```
$ scancel <jobid>
```

Monitorowanie stanu systemu i zadań

Lista dostępnych węzłów i ich stan

Listę dostępnych węzłów sprawdzamy poleceniem `sinfo`

```
login@svradmin:~$ sinfo -p hgx
```

```
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
```

```

hgx*          up    infinite    1    down* obl1
hgx*          up    infinite    1          mix hgx2
hgx*          up    infinite    2    idle hgx1,obl2

```

Sprawdzenie stanu kolejki

Stan bieżący kolejki sprawdzamy poleceniem `squeue` (parametr `-u` umożliwia wskazanie użytkownika, a parametr `-t` lub `--states` wskazanie interesujących nas stanów zadań), przykład (dla użytkownika *mily*):

```

mily@svradmin:~$ squeue -u mily -t all

```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES
NODELIST (REASON)						
53929	hgx	hostname	mily	CD	0:00	1

hgx1

```

mily@svradmin:~$ squeue -u mily

```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES
NODELIST (REASON)						
53930	hgx	bash	mily	R	0:05	
1 hgx1						

w przypadku braku zadań oczekujących lub uruchomionych wynik polecenia `squeue` jest pusty:

```

mily@svradmin:~$ squeue -u mily

```

JOBID	PARTITION	NAME	USER	ST	TIME	NODES
NODELIST (REASON)						

Kolumna ST oznacza status zadania: <https://curc.readthedocs.io/en/latest/running-jobs/squeue-status-codes.html>

Polecenie `squeue` może dostarczyć także informacje o stanie kolejki w bardziej rozbudowanej formie:

```

mily@svradmin:~$ squeue -u mily -l
mily@svradmin:~$ squeue -u mily -t all -o%all

```

Sprawdzanie historii zadań

Do sprawdzania historii zleconych zadań służy polecenie `sacct` - podany poniżej przykład wyświetli zadania rozpoczęte po godzinie 13:04 dnia 21.05.2023r:

```

mily@svradmin:~$ sacct --starttime 2023-05-21T13:04:00 --
format=User,JobID

```

User	JobID

mily	53930
	53930.extern
	53930.0

W przypadku chęci uzyskania większej liczby informacji należy wszystkie interesujące nas pola dodać do parametru `format`:

```

mily@svradmin:~$ sacct --starttime 2023-05-21T13:04:00 \
--format=User,JobID,Jobname%50,partition,state,time,start,end,elapsed,\
MaxRss,MaxVMSize,nnodes,ncpus,nodelist

```

Wyświetlanie limitów zasobowych

Polecenie `sacctmgr`

Do wyświetlenia limitów zasobowych można się posłużyć poleceniem `sacctmgr` np. w taki sposób (przykład dla użytkownika o loginie *mily*):

```
mily@svradmin:~$ sacctmgr list associations tree \
format=account,user,grptres%30 users=mily
Account                                User                                GrpTRES
-----
ai                                     mily                                cpu=2
```

i jeszcze przykład dla użytkownika przypisanego do kilku kont rozliczeniowych (Accounts):

```
mily@svradmin:~$ sacctmgr list associations tree \
format=account,user,grptres%30 users=ppoznaniak
Account                                User                                GrpTRES
-----
test3                                 ppoznaniak                        cpu=2,gres/gpu=2,mem=1G
test2                                 ppoznaniak                        cpu=3,gres/gpu=2,mem=1G
ai                                    ppoznaniak                        cpu=3,gres/gpu=2,mem=1G
```

Polecenie scontrol (dla zaawansowanych)

Do wyświetlania statusów demona zarządzającego slurmctld służy polecenie scontrol. Za jego pomocą można np. wyświetlić limity zasobowe przydzielone i zużyte (wartość zmiennej GrpTRES w wynikach):

```
$ scontrol show assoc_mgr users=LOGIN account=KONTO-ROZLICZENIOWE \
flags=assoc
```